AES (Advanced Encription Standard)

- Cifra de Bloco

Tamanho da Chave (bits)	Nº de rodadas
128	10
192	12
256	14

Plain Text : 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF

State inicial

00	44	88	CC
11	55	99	DD
22	66	AA	EE
33	77	BB	FF

Como funciona ?

Plain Text

↓

AddRoundKey ← K0

↓

Rounds 1 a 9 :

SubBytes

ShiftRows

MixColumns

AddRoundKey

↓

Round 10 :

SubBytes

ShiftRows

AddRoundKey

↓

CipherText

Made with Goodnotes

- 1ª operação: AddRoundKey

Secret Key: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

O AddRoundKey combina State com uma chave de rodada por meio de XOR.

00 XOR 00 = 00

11 XOR 01 = 10

22 XOR 02 = 20

. . .

State após AddRoundKey inicial:

$$\begin{bmatrix} 00 & 40 & 80 & C0 \\ 10 & 50 & 90 & D0 \\ 20 & 60 & A0 & E0 \\ 30 & 70 & B0 & F0 \end{bmatrix}$$

OBS: O AddRoundKey é reversível:

$(A \oplus K) \oplus K = A$

- 2ª operação: SubBytes

O SubByte pega cada byte da matriz e o substitui por outro byte, usando uma tabela fixa chamada S-Box

Exemplo:

S(00) = 63 A S-Box tem 256 entradas, uma para cada possível valor de byte, de 00 até FF.

S(10) = CA

S(20) = B7

S(30) = 04

A característica principal do SubBytes é que ele é não-linear.

$S(A \oplus B) \neq S(A) \oplus S(B)$

- 3ª operação: ShiftRows

O ShiftRows reorganiza os bytes da matriz.

Ele desloca as linhas para a esquerda.

- Linha 0: não desloca.
- Linha 1: desloca 1 posição.
- Linha 2: desloca 2 posições.
- Linha 3: desloca 3 posições.

ANTES:

$$\begin{bmatrix} 63 & 09 & CD & BA \\ CA & 53 & 60 & 70 \\ B7 & D0 & E0 & E1 \\ 04 & 51 & E7 & 8C \end{bmatrix}$$

DEPOIS:

$$\begin{bmatrix} 63 & 09 & CD & BA \\ 53 & 60 & 70 & CA \\ E0 & E1 & B7 & D0 \\ 8C & 04 & 51 & E7 \end{bmatrix}$$

- 4ª operação: MixColumns

O MixColumns pega cada coluna de quatro bytes e produz uma nova coluna.

Primeira coluna que entra no MixColumns:

$$\begin{bmatrix} 63 \\ 53 \\ E0 \\ 8C \end{bmatrix} = a$$

$$\begin{bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a0 \\ a1 \\ a2 \\ a3 \end{bmatrix}$$

$$b0 = (02 \cdot 63) \oplus (03 \cdot 53) \oplus E0 \oplus 8C$$

Made with Goodnotes

Piret & Quisquater

O ataque de Piret usa as diferenças provocadas por uma falha para descobrir quais valores da chave AES são possíveis.

Round 9
SubBytes
ShiftRows
Injeção de Falha
Um byte do State é alterado
MixColumns
AddRoundKey k9

Exemplo: Depois dos rounds 1 a 8, chegamos ao round 9.

Após SubBytes e ShiftRows, o State é:

$$\begin{bmatrix} 54 & 6B & 96 & A1 \\ D9 & A0 & BB & 11 \\ 90 & 9A & F4 & 70 \\ A1 & B5 & 0E & 2F \end{bmatrix}$$

Alteramos o primeiro byte da primeira coluna

$$\begin{bmatrix} 55 \\ D9 \\ 90 \\ A1 \end{bmatrix}.$$

Round 9 - MixColumns

Aplicamos MC nas duas execuções.

Byte	AES correto	AES defeituoso	Diferença
0	E9	EB	02
1	F7	F6	01
2	4E	4F	01
3	EC	EF	03

$\Delta y = (02, 01, 01, 03)$

Round 9 - AddRoundKey k9

Depois do ARK do round 9, os quatro bytes são:

Byte	Correto	Defeituoso	Diferença
0	BD	BF	02
1	6E	6F	01
2	7C	7D	01
3	3D	3E	03

A diferença segue igual, pois a mesma k9 é aplicada nas duas execuções

Round 10 - SB, SR, ARK

O último round não possui MixColumns.

As diferenças passam pela S-Box, que é não linear, e o SR move os quatro bytes para posições diferentes.

No final temos:

Cipher Text correto

69C4E0D86A7B0430D8CDB7807B4C55A

Cipher Text defeituoso — 4 bytes diferentes

1BC4E0D86A7B04A5D8CD588070B3C55A

Made with Goodnotes

O atacante tenta recuperar K10.

O atacante usa a fórmula para cada byte afetado.

$\delta_i = S^{-1}(C_i \oplus k_i) \oplus S^{-1}(F_i \oplus k_i)$

Cada termo:

C_i => byte do ciphertext correto.

F_i => byte do ciphertext defeituoso.

k_i => byte de K10 que estamos tentando descobrir.

S^{-1} => S-Box inversa.

δ_i => diferença reconstruída antes do último SB.

De onde vem os candidatos de K10 ?

Cada byte possui 8 bits, portanto 256 valores possíveis.
(00, ..., FF)

Como uma falha no MC do round 9 afeta quatro bytes do ciphertext, precisamos descobrir quatro bytes de K10 simultaneamente.

Por exemplo, para a primeira coluna atacada, as posições de K10 são:

K10[0], K10[13], K10[10], K10[7]

Sem nenhuma informação adicional, existem:

$256^4 = 2^{32} = 4.294.967.296$ combinações possíveis!

Primeiro construímos a lista D

Lembra que uma falha 01 na primeira posição gera a coluna:

(02, 01, 01, 03)

Mas falha pode ter qualquer valor de 01 a FF e ocorrer em qualquer uma das quatro linhas.

Por isso, construímos uma lista de todos os padrões possíveis:

$D = \{ MixColumns(\Delta X) \}$, em que ΔX possui exatamente um byte nulo.

Existem: $4 \cdot 255 = 1020$ padrões possíveis de 4 bytes.

Como o programa testa um byte de K10 ?

Na posição 0, temos:

$C_0 = 69$, $F_0 = 1B$

O programa testa todos os valores de k, de 00 até FF, calculando:

$\delta_0(k) = S^{-1}(69 \oplus k) \oplus S^{-1}(1B \oplus k)$

Por exemplo, para o candidato 13:

$69 \oplus 13 = 7A$

$1B \oplus 13 = 08$

Aplicando a S-Box inversa:

$S^{-1}(7A) = BD$

$S^{-1}(08) = BF$

Então:

$\delta_0(13) = BD \oplus BF = 02$

Portanto, o candidato 13 produz a diferença 02.

Mas outros valores de chave podem produzir essa mesma diferença.

O programa faz isso para os 256 candidatos e cria uma tabela.

Made with Goodnotes

Como combinamos os quatro bytes?

Agora temos quatro tabelas, uma para cada posição do ciphertext.

Para cada padrão da lista D, buscamos nas tabelas quais bytes de chave produzem as diferenças exigidas.

Por exemplo, para o padrão:

$$D_i = (02, 01, 01, 03)$$

Precisamos encontrar:

$$\begin{cases} S_0(k_0) = 02 \\ S_{13}(k_{13}) = 01 \\ S_{10}(k_{10}) = 01 \\ S_7(k_7) = 03 \end{cases}$$

Se encontrarmos valores que satisfazem as quatro equações, guardamos a combinação:

$$(k_0, k_{13}, k_{10}, k_7)$$

No exemplo, a combinação correta é:

$$(13, 2B, A7, 17)$$

Ela produz:

$$(02, 01, 01, 03)$$

Portanto, é compatível com a falha.

MAS existem outras combinações compatíveis. Uma única falha não identifica necessariamente a chave correta.

Como eliminamos os candidatos errados?

Imagine que, após a primeira falha, o programa tenha encontrado os seguintes candidatos fictícios:

13 2B A7 17 ✓

42 10 8C 55 ✓

A1 73 20 9F ✓

CC 51 12 30 ✓

Agora injetamos outra falha no mesmo grupo de quatro bytes e obtemos um novo ciphertext F2.

Repetimos todo o processo.

Suponha que F2 aceite apenas:

13 2B A7 17 ✓

78 30 B4 11 ✓

CC 51 12 30 ✓

O programa calcula a interseção:

$$L = L_1 \cap L_2$$

Restam:

13 2B A7 17

CC 51 12 30

Uma terceira falha pode eliminar o segundo candidato, deixando:

Quatro bytes recuperados de k10: 13 2B A7 17 ✓

Made with Goodnotes

Como recuperamos a chave K10 inteira?

Até agora, recuperamos apenas 4 bytes, mas K10 tem 16 bytes.

Precisamos repetir o ataque em cada uma das quatro colunas do State antes do Mix Columns do round 9.

Cada coluna afetada correspondente a um grupo diferente de quatro posições no ciphertext:

Coluna atacada	Posições de K10 recuperadas
0	0, 13, 10, 7
1	4, 1, 14, 11
2	8, 5, 2, 15
3	12, 9, 6, 3

Assim, depois de obter falhas suficientes nas quatros colunas, recuperamos todos os bytes de K10.

A última chave de rodada do nosso exemplo é:

K10 recuperada

13	E3	F3	4D
11	94	07	2B
1D	4A	A7	30
7F	17	8B	C5

K10 = 13111D7FE3944A17F307A78B4D2B30C5

Como transformamos K10 na chave original K0?

O AES-128 usa um algoritmo chamado KEY EXPANSION, que transforma K0 em onze chaves.

K0 → K1 → ... → K10

O processo é reversível. Portanto, se recuperarmos K10 inteira, podemos executar o Key Expansion ao contrário.

Key Expansion

Por que o AES precisa de Key Expansion?

No AES-128, temos uma chave original de 128 bits, ou 16 bytes.

No nosso exemplo:

K0 = 000102030405060708090A0B0C0E0F

Mas o AES utiliza uma chave diferente em cada AddRoundKey:

- K0: AddRoundKey inicial.
- K1 a K9: AddRoundKey dos rounds 1 a 9.
- K10: AddRoundKey do último round.

São 11 chaves de 16 bytes, totalizando 176 bytes de material de chave expandida.

O Key Expansion é o algoritmo determinístico que gera essas chaves a partir de K0. Não são 11 chaves secretas independentes.

Primeiro dividimos a chave em quatro palavras

Cada palavra (word) contém 4 bytes, ou 32 bits.

A chave original fica:

K0 - chave original

W0	W1	W2	W3
00010203	04050607	08090A0B	0C0D0E0F

A partir dessas quatro palavras, o AES calcula W4, W5, W6, W7, formando K1.

Depois calcula W8 a W11, formando K2, e assim sucessivamente.

No total, gera 44 palavras, de W0 a W43.

Made with Goodnotes

As quatro operações do Key Expansion

O algoritmo utiliza quatro operações principais.

A. RotWord - rotacionar os bytes

Pega uma palavra de quatro bytes e move o primeiro byte para o final.

Por exemplo:

0C 0D 0E 0F ⟹ 0D 0E 0F 0C

B. SubWord - aplicar a S-Box

Aplica a mesma S-Box do SubBytes a cada um dos quatro bytes.

No nosso exemplo:

SubWord (0D 0E 0F 0C)

As substituições são:

Byte original	S-Box
0D	D7
0E	AB
0F	76
0C	FE

Resultado = D7 AB 76 FE

C. Rcon - constante de rodada

O Rcon é uma constante utilizada a cada quatro palavras.

Para calcular a primeira chave derivada, usamos:

Rcon[1] = 01 00 00 00

Para as próximas etapas:

Etapa	Rcon
k1	01 00 00 00
k2	02 00 00 00
k3	04 00 00 00
k4	08 00 00 00
k5	10 00 00 00
k6	20 00 00 00
k7	40 00 00 00
k8	80 00 00 00
k9	1B 00 00 00
k10	36 00 00 00

O Rcon ajuda a diferenciar as transformações aplicadas nas etapas de expansão.

D. XOR - combinar as palavras

A última operação é o XOR.

Lembre-se:

$$A \oplus B \oplus B = A$$

Essa propriedade vai permitir inverter o Key Expansion.

Made with Goodnotes

Calculando k1 inteira a partir de k0

Temos:

w0 = 00010203

w1 = 04050607

w2 = 08090A0B

w3 = 0C0D0E0F

A primeira palavra nova é calculada por:

$w4 = w0 \oplus g(w3)$

A função g é:

$g(w) = \text{SubWord}(\text{RotWord}(w)) \oplus \text{Rcon}$

Calculando:

Passo 1 - RotWord:

w3 = 0C 0D 0E 0F

RotWord(w3) = 0D 0E 0F 0C

Passo 2 - SubWord:

RotWord(w3) = 0C 0D 0E 0F

SubWord(RotWord(w3)) = D7 AB 76 FE

Passo 3 - XOR com Rcon[1]:

D7 AB 76 FE

$\oplus$ 01 00 00 00

D6 AB 76 FE

Portanto:

g(w3) = D6AB76FE

Passo 4 - XOR com w0:

00 01 02 03

$\oplus$ D6 AB 76 FE

D6 AA 74 FD

Logo:

w4 = D6AA74FD é a primeira palavra do k1.

Calculando as outras três palavras:

Basta fazer XOR com a palavra anterior da chave original e a palavra recém-gerada.

w5 = w1 ⊕ w4	w6 = w2 ⊕ w5	w7 = w3 ⊕ w6
04 05 06 07	08 09 0A 0B	0C 0D 0E 0F
⊕ D6 AA 74 FD	⊕ D2 AF 72 FA	⊕ DA A6 78 F1
D2 AF 72 FA	DA A6 78 F1	D6 AB 76 FE

Portanto, k1 é:

w4	w5	w6	w7
D6AA74FD	D2AF72FA	DAA678F1	D6AB76FE

k1 = D6AA74FDD2AF72FADAA678F1D6AB76FE

A regra geral é:

$$w_i = \begin{cases} w_{i-4} \oplus g(w_{i-1}), & i \bmod 4 = 0 \\ w_{i-4} \oplus w_{i-1}, & \text{caso contrário} \end{cases}$$

Made with Goodnotes

Como recuperar K0 a partir de K10?

Por que é possível inverter o Key Expansion?

Porque todas as operações utilizadas possuem inversas, e as relações entre as palavras permitem calcular as anteriores a partir das posteriores.

Na prática, nem precisamos aplicar a S-Box inversa para desfazer o Key Expansion.

Basta usar a mesma função g, mas reorganizar as equações.

Por exemplo, se:

$$w5 = w1 \oplus w4$$

então:

$$w1 = w5 \oplus w4$$

Começamos pela última chave K10

No nosso exemplo oficial AES-128:

K10 = 13111D7FE3944A17F307A78B4D2B30C5

K10

w40	w41	w42	w43
13111D7F	E3944A17	F307A78B	4D2B30C5

Queremos recuperar as quatro palavras anteriores:

K9 = (w36, w37, w38, w39)

O segredo é começar da direita para a esquerda.

Primeiro recuperamos w39

Na expansão original:

$$w43 = w39 \oplus w42$$

Portanto, invertendo:

$$w39 = w43 \oplus w42$$

$$w39 = BE2C974E$$

Depois recuperamos w38 e w37

$$w38 = w42 \oplus w41 \qquad w37 = w41 \oplus w40$$

$$w38 = 1093ED9C \qquad w37 = F0855768$$

K9 reconstruída

w36	w37	w38	w39
549932D1	F0855768	1093ED9C	BE2C974E

K9 = 549932D1F0855768 1093ED9C BE2C974E

Recuperando w36 - a parte mais importante

Na expansão original, a relação era:

$$w40 = w36 \oplus g(w39)$$

Portanto:

$$w36 = w40 \oplus g(w39)$$

Já sabemos:

$$w40 = 13111D7F$$

$$w39 = BE2C974E$$

Precisamos calcular g(w39).

RotWord → SubWord → XOR com Rcon[10] → g(w39) = 47882FAE

BE 2C 97 4E → 2C 97 4E BE → 71 88 2F AE → Rcon[10] = 36 00 00 00 → 47 88 2F AE

w36 = w40 ⊕ g(w39)

w36 = 549932D1

Made with Goodnotes

SM4

Dados iniciais:

Plaintext - 128 bits = 0123456789ABCDEFFEDCBA9876543210

Chave original - 128 bits = 0123456789ABCDEFFEDCBA9876543210

Ciphertext = 681EDF34D206965E86B3E94F536E4246

Como o SM4 organiza os dados?

No SM4, dividimos os 128 bits em quatro palavras de 32 bits:

$$P = X_0 \| X_1 \| X_2 \| X_3$$

O símbolo $\|$ significa concatenação.

X_0 = 01234567

X_1 = 89ABCDEF

X_2 = FEDCBA98

X_3 = 76543210

Cada palavra contém quatro bytes. Por exemplo:

X_0 = 01 23 45 67

A fórmula de uma rodada

O SM4 executa 32 rodadas.

Em cada rodada, ele calcula uma nova palavra:

$$X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)$$

Aqui:

- X_i é a palavra mais antiga das quatro.
- rk_i é a chave de rodada de 32 bits.
- T é uma transformação composta pela S-Box e por uma transformação linear.

A função T pode ser dividida em:

$$T(A) = L(\tau(A))$$

Em que:

$\tau(A)$ = aplicar S-Box nos quatro bytes.

e

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

O símbolo $\lll$ indica rotação circular para a esquerda, não um deslocamento que descarta os bits.

Nosso objetivo na primeira rodada é calcular X_4.

Antes da primeira rodada: gerar rk0

Dividimos a chave original em quatro palavras:

MK_0 = 01234567

MK_1 = 89ABCDEF

MK_2 = FEDCBA98

MK_3 = 76543210

Made with Goodnotes

O SM4 possui quatro constantes iniciais chamadas Fk:

Constante	Valor
FK0	A3B1BAC6
Fk1	56AA3350
Fk2	677D9197
Fk3	B27022DC

Calculamos:

$k_i = MK_i \oplus Fk_i$

Por exemplo:

$k_0 = 01234567 \oplus A3B1BAC6 = A292FFA1$

Aplicando às outras três palavras:

Palavra	MK	FK	Resultado
K0	01234567	A3B1BAC6	A292FFA1
k1	89ABCDEF	56AA3350	DF01FEBF
k2	FEDCBA98	677D9197	99A12B0F
k3	76543210	B27022DC	C42410CC

Esses quatro valores são o estado inicial da expansão de chave.

A primeira chave de rodada é calculada por:

$rk_0 = K_4$

com:

$k_4 = k_0 \oplus T'(k_1 \oplus k_2 \oplus k_3 \oplus Ck_0)$

A transformação T' utiliza a S-Box do SM4 e a função linear:

$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$

A constante da primeira rodada de expansão é:

$Ck_0 = 00070E15$

Calculando a primeira chave de rodada rk0

Temos:

$k_4 = k_0 \oplus T'(k_1 \oplus k_2 \oplus k_3 \oplus Ck_0)$

Passo 1 - XOR das três palavras e da constante

$A = DF01FEBF \oplus 99A12B07 \oplus C42410CC \oplus 00070E15 = 8283CB69$

Passo 2 - Aplicar a S-Box

Separamos A em quatro bytes:

82 83 CB 69

Consultamos a S-Box do SM4:

Entrada	Saída
82	8A
83	D2
CB	41
69	22

Portanto:

$B = \tau(A) = 8AD24122$

Made with Goodnotes

Passo 3 - Aplicar a transformaçoo linear L'

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

Calculando as rotações circulares:

$B = 8AD24122$

$B \lll 13 = 4824515A$

$B \lll 23 = 91546920$

Esses valores devem ser combinados com XOR. O resultado é:

$$L'(B) = 53B37958$$

Passo 4 - XOR com ko interno

$$K_4 = A292FFA1 \oplus 53B37958$$

$$rk_0 = K_4 = F12186F9$$

Essa é a primeira chave de rodada que entra na cifragem do SM4.

Observe a diferença em relação ao AES: a chave de rodada SM4 tem 4 bytes, e não 16 bytes.

Agora vamos cifrar: rodada 0 completa

Temos os quatro valores iniciais:

$X_0 = 01234567$

$X_1 = 89ABCDEF$

$X_2 = FEDCBA89$

$X_3 = 76543210$

$rk_0 = F12186F9$

A fórmula é:

$$X_4 = X_0 \oplus L(\tau(X_1 \oplus X_2 \oplus X_3 \oplus rk_0))$$

Passo 1 - XOR das três palavras com rko

$$A = X_1 \oplus X_2 \oplus X_3 \oplus rk_0 = F002C39E$$

Passo 2 - Aplicar a S-Box do SM4

Separamos A em quatro bytes:

$$A = (F0, 02, C3, 9E)$$

Cada byte é substituído independentemente:

Byte de entrada	S-Box SM4
F0	18
02	E9
C3	92
9E	B1

Portanto:

$$B = \tau(A) = 18E992B1$$

Perceba que a S-Box recebe 32 bits, mas internamente faz quatro substituição de 8 bits.

Made with Goodnotes

Passo 3 - Aplicar a transformação linear L

Agora calculamos:

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

Operação	Resultado
B	18E992B1
B <<< 2	63A64AC4
B <<< 10	A64AC463
B <<< 18	4AC463A6
B <<< 24	B118E992

Fazer XOR das cinco parcelas

Agora calculamos:

$L(B) = 18E992B1 \oplus 63A64AC4 \oplus A64AC463 \oplus 4AC463A6 \oplus B118E992$

$L(B) = 26D99622$

Aplicar o XOR final com X_0

$X_4 = X_0 \oplus L(B)$

$X_4 = 01234567 \oplus 26D99622$

$X_4 = 27FAD345$

DFA no SM4

No nosso exemplo SM4:

Plaintext = 0123456789ABCDEFFEDCBA9876543210

Chave secreta = 0123456789ABCDEFFEDCBA9876543210

Ciphertext = 681EDF34D206965E86B3E94F536E4246

No SM4, o ciphertext é organizado assim:

$C = (X_{35}, X_{34}, X_{33}, X_{32})$

Portanto, a partir do ciphertext, o atacante já conhece:

$X_{35} = 681EDF34$

$X_{34} = D206965E$

$X_{33} = 86B3E94F$

$X_{32} = 536E4246$

No SM4, três das quatro palavras que entram na última rodada já aparecem diretamente no ciphertext.

A última rodada que queremos atacar

A rodada 31 calcula:

$$X_{35} = X_{31} \oplus L(\tau(X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}))$$

Para facilitar, chamamos a entrada da S-Box de:

$$A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}$$

E a saída da S-Box de:

$$B = \tau(A)$$

Então:

$$X_{35} = X_{31} \oplus L(B)$$

Made with Goodnotes

Vamos injetar uma falha real no nosso exemplo

Para esse experimento, vamos supor que conseguimos alterar o primeiro byte de A, imediatamente antes das S-Boxes da última rodada.

Escolhemos uma falha XOR de valor 01.

Para conferir os cálculos, a execução correta possui estes valores internos:

X_{31} = 7B938F4C

A = 96FF9D45

rk_{31} = 9124A012

Esses valores são conhecidos pelo simulador, mas não pelo atacante.

A falha altera:

A correto = 96 FF 9D 45

Falha XOR = 01 00 00 00

A defeituoso = 97 FF 9D 45

Agora vamos acompanhar as duas execuções.

Passagem pelas S-Boxes

Aplicamos a S-Box do SM4 aos quatro bytes.

Byte	Entrada correta	Entrada defeituosa	S-Box correta	S-Box defeituosa
0	96	97	1A	55
1	FF	FF	48	48
2	9D	9D	8C	8C
3	45	45	73	73

Portanto:

B = 1A488C73

B' = 55488C73

A diferença após a S-Box é:

$\Delta B = B \oplus B'$

ΔB = 4F000000

Obs: A falha ocorreu em um byte, e somente uma das quatro S-boxes produziu uma saída diferente.

Passagem pela transformação linear L

A transformação linear do SM4 é:

$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$

Para a execução correta:

L(1A488C73) = 138D5078

Como L é linear, podemos calcular a diferença diretamente:

$\Delta L = L(\Delta B)$

$\Delta L = L(4F000000)$

ΔL = 734E3D3D

Finalmente, a última rodada faz XOR com X_{31}:

$X_{35} = X_{31} \oplus L(B)$

Na execução correta:

7B938F4C ⊕ 138D5078 = 681EDF34

Na execução defeituosa, a diferença 734E3D3D altera X_{35}:

X'_{35} = 681EDF34 ⊕ 734E3D3D = 1B50E209

Made with Goodnotes

Os dois ciphertexts que o atacante recebe

Ciphertext correto C

681EDF34 D206965E 86B3E94F 536E4246

Ciphertext defeituoso f

1B50E209 D206965E 86B3E94F 536E4246

Diferença C XOR f

734E3D3D 00000000 00000000 00000000

Agora vamos esquecer que conhecemos A = 96ff9D45 e rk31 = 9124A012.

A partir daqui, vamos agir como atacante.

Como o atacante descobre o primeiro byte de rk_{31}?

O atacante observa:

$$\Delta X_{35} = 734E3D3D$$

Ele sabe que essa diferença veio da transformação L:

$$\Delta X_{35} = L(\Delta B)$$

Então aplica a transformação inversa:

$$\Delta B = L^{-1}(734E3D3D)$$

Resultado:

$$\Delta B = 4F000000$$

Isso significa que, na primeira S-Box:

$$S(a) \oplus S(a \oplus 01) = 4F$$

O atacante conhece o valor da falha 01 neste experimento e agora testa todos os 256 valores possíveis de "a".

Por exemplo:

- Testa a = 00: calcula $S(00) \oplus S(01)$ e compara com 4F.
- Testa a = 01: calcula $S(01) \oplus S(00)$ e compara com 4F.
- Continua até a = ff.

Para esta observação, somente dois valores satisfazem a equação:

$$a \in \{96, 97\}$$

Portanto, reduzimos os 256 valores possíveis da entrada da S-Box para apenas dois!

Transformando os candidatos em bytes de chave

Lembre-se:

$$A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}$$

Então:

$$rk_{31} = A \oplus X_{32} \oplus X_{33} \oplus X_{34}$$

O atacante conhece as três palavras do ciphertext:

$$X_{32} \oplus X_{33} \oplus X_{34} = 07DB3D57$$

O primeiro byte dessa combinação é 07.

Logo:

$$k = a \oplus 07$$

Para os dois candidatos:

$$96 \oplus 07 = 91$$

$$97 \oplus 07 = 90$$

Temos:

$rk_{31}[0] \in \{91, 90\}$

Uma única falha reduziu os candidatos do primeiro byte de chave de 256 para dois, dentro do modelo de falha conhecido.

Made with Goodnotes

Fazemos uma segunda falha

Agora repetimos a cifragem com o mesmo plaintext, mas alterarmos o mesmo byte de A usando XOR 02, em vez de 01.

A nova equação diferencial é:

$$S(a) \oplus S(a \oplus 02) = d_2$$

Para essa execução, o valor observado após desfazer L é:

$$d_2 = 81$$

Testamos novamente os candidatos e intersectamos os resultados.

Experimento	Falha conhecida	Diferença após S-Box	Candidatos de entrada
1	01	4F	96, 97
2	02	81	Conjunto compatível com a 2ª opção
Interseção	Ambas	Ambas	96

Agora sabemos:

$a = 96$

E:

$rk_{31}[0] = 96 \oplus 07$

$rk_{31}[0] = 91$

Recuperamos o primeiro byte da última subchave!

Recuperando os quatro bytes de rk_{31}

Até agora, atacamos o primeiro byte da entrada da S-Box.

Para recuperar a palavra inteira, repetimos o experimento nas outras três posições de byte.

No nosso exemplo, usando falhas XOR 01 e 02 separadamente em cada posição, obtemos:

Byte	Diferença com falha 01	Diferença com falha 02	Entrada A recuperada	Byte de rk_{31}
0	4f	81	96	91
1	71	83	ff	24
2	49	6f	9D	A0
3	80	C9	45	12

Portanto:

$A = 96ff9D45$

Já sabemos que:

$X_{32} \oplus X_{33} \oplus X_{34} = 07DB3D57$

Calculamos:

$$\begin{array}{r} 96\ \ ff\ \ 9D\ \ 45 \\ \oplus\ 07\ \ DB\ \ 3D\ \ 57 \\ \hline 91\ \ 24\ \ A0\ \ 12 \end{array} \qquad rk_{31} = 9124A012$$

Made with Goodnotes

Como recuperar o resto das chaves?

Aqui existe uma diferença entre recuperar a última subchave e recuperar a chave inteira.

Depois de conhecer rk31, podemos calcular a palavra X_{31}:

$$X_{31} = X_{35} \oplus T(X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31})$$

No nosso exemplo:

$$X_{31} = 7B938F4C$$

Agora temos informação suficiente para reconstruir o estado na entrada da rodada 31.

Mas ainda não conhecemos rk30.

Para recuperá-la, precisamos de outras observações de falhas que restrinjam os valores dessa subchave. O mesmo vale para rk29 e rk28.

Uma estratégia didática é repetir o procedimento com falhas controladas nas entradas das funções T das rodadas 30, 29 e 28. Porém, para essas rodadas, o atacante também precisa reconstruir ou restringir os estados internos necessários; não basta copiar diretamente os quatro bytes do ciphertext.

Para o nosso vetor oficial, as quatro últimas subchaves verdadeiras são:

Rodada	SubChave
28	428D3654
29	62293496
30	01CF72E5
31	9124A012

Esses valores são os resultados que um ataque completo deve recuperar, não resultados que demonstramos ter extraído dos ciphertexts anteriores. Até aqui, nossa demonstração recuperou efetivamente rk31 no modelo de falha controlada.

Como obter a chave original a partir dessas quatro subchaves?

Agora entramos no Inverse key Expansion do SM4.

$$rk_i = k_{i+4}$$

Então:

$$k_{32} = 428D3654$$
$$k_{33} = 62293496$$
$$k_{34} = 01CF72E5$$
$$k_{35} = 9124A012$$

A fórmula original da expansão é:

$$k_{i+4} = k_i \oplus T'(k_{i+1} \oplus k_{i+2} \oplus k_{i+3} \oplus CK_i)$$

Para inverter:

$$k_i = k_{i+4} \oplus T'(k_{i+1} \oplus k_{i+2} \oplus k_{i+3} \oplus CK_i)$$

Exemplo: recuperar k31

Começamos pela última palavra:

$$k_{31} = k_{35} \oplus T'(k_{32} \oplus k_{33} \oplus k_{34} \oplus CK_{31})$$

A constante é: $CK_{31} = 646B7279$

Calculamos o XOR: $U = 428D3654 \oplus 62293496 \oplus 01CF72E5 \oplus 646B7279$

Depois aplicamos T', que utiliza a S-Box do SM4 e a transformação Linear:

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

O resultado é: $T'(U) = 605CAC93$

Então: $k_{31} = 9124A012 \oplus 605CAC93 = F1780C81$

Made with Goodnotes

Recuperamos a palavra anterior da expansão!

Continuamos da mesma maneira, em ordem decrescente:

$K_{30}, K_{29}, \cdots, K_0$

Quando recuperamos K_0, K_1, K_2, K_3, desfazemos as constantes FK:

$MK_i = K_i \oplus FK_i$

O resultado final do vetor de teste é:

MK = 0123456789ABCDEFFEDCBA9876543210

Giraud no AES-128

Explora uma falha no estado próximo ao início da última rodada, permitindo analisar a relação entre o ciphertexts correto e defeituoso e a última chave de rodada.

Vamos usar o exemplo oficial AES-128

Plaintext: 00112233445566778899AABBCCDDEEFF

Chave k0: 000102030405060708090A0B0C0D0E0F

Ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A

Onde Giraud injeta a falha?

Exige uma falha de um único bit em um byte na entrada do último SubBytes, depois do AddRoundKey da rodada 9.

Rounds 1 a 9 - AES normal

↓

AddRoundKey K9

↓

Injeção de falha de 1 bit

Antes do SubBytes do round 10

↓

SubBytes

↓

ShiftRows

↓

AddRoundKey k10

↓

Ciphertext defeituoso

Lembre-se de que o último round do AES não possui MixColumns

Por isso, quando alteramos um único byte nessa posição, apenas um byte do ciphertext é afetado. O ShiftRows muda a posição desse byte, mas não espalha a diferença para outros bytes.

Vamos injetar uma falha real

Usaremos o primeiro byte do State na entrada do SubBytes do round 10.

No nosso vetor AES, esse byte vale:

a = BD

Vamos alterar somente seu bit menos significativo:

e = 01

Made with Goodnotes

A execução defeituosa recebe:

$a' = a \oplus e$

$BD \oplus 01 = BC$

AES Correto	AES com falha
BD	BC
10111101	10111100

O que acontece no último SubByte?

O AES aplica sua S-Box aos dois valores.

Execução	Entrada S-Box	Saída S-Box
Correta	BD	7A
Defeituosa	BC	65

A diferença da S-Box é:

$7A \oplus 65 = 1F$

A diferença original era 01, mas depois da S-Box virou 1f.

O que acontece no ShiftRows?

Escolhemos o byte na linha 0, coluna 0.

O ShiftRows desloca a linha 0 em zero posições. Portanto, esse byte continua na posição 0.

Depois vem o último AddRoundKey, com k10.

O byte correspondente da chave k10 do nosso exemplo é:

$k10[0] = 13$

Na execução correta:

$C_0 = 7A \oplus 13 = 69$

Na execução defeituosa:

$F_0 = 65 \oplus 13 = 76$

Assim, temos dois ciphertexts:

Ciphertext correto

69C4E0D86A7B0430D8CDB78070B4C55A

Ciphertext defeituoso

76C4E0D86A7B0430D8CDB78070B4C554

Diferença

1F000000000000000000000000000000

O atacante observa que apenas o byte 0 mudou:

$C_0 = 69$ $F_0 = 76$

Como o Giraud encontra os candidatos de k10?

Aqui está o coração do ataque.

O atacante não sabe que o byte da chave é 13. Ele precisa descobrir esse valor.

Sabemos que:

$C_0 = S(a) \oplus k$

$F_0 = S(a \oplus e) \oplus k$

Em que k é o byte secreto de k10.

Made with Goodnotes

Como desfazemos o XOR com a chave?

Aplicando XOR com o mesmo candidato K:

$$C_0 \oplus k = S(a)$$

Depois, aplicamos a S-Box inversa:

$$S^{-1}(C_0 \oplus k) = a$$

Fazemos o mesmo com F:

$$S^{-1}(F_0 \oplus k) = a \oplus e$$

E calculamos a diferença:

$$\delta(k) = S^{-1}(C_0 \oplus k) \oplus S^{-1}(F_0 \oplus k)$$

Se a chave candidata estiver correta, δ precisa ter exatamente um bit igual a 1.

Isso significa que ela deve pertencer ao conjunto:

$$D = \{01, 02, 04, 08, 10, 20, 40, 80\}$$

Esses são os oito valores possíveis de uma falha de um único bit.

Testando o candidato 13

Vamos calcular com os valores reais:

$C_0 = 69$

$F_0 = 76$

Testamos:

$k = 13$

Passo A - Desfazer o último AddRoundKey

$69 \oplus 13 = 7A$

$76 \oplus 13 = 65$

Passo B - Aplicar a S-Box inversa

$S^{-1}(7A) = BD$

$S^{-1}(65) = BC$

Passo C - Calcular a diferença

$\delta(13) = BD \oplus BC = 01$

Em binário: 00000001

Tem exatamente um bit igual a 1.

Conclusão: o candidato 13 é compatível com a falha observada.

Mas ainda não sabemos se ele é o único.

O computador testa os 256 candidatos

Cada byte de k10 pode valer de 00 a ff.

O programa calcula $\delta(k)$ para todos eles e guarda apenas os que produzem uma diferença de um bit.

Para o nosso exemplo real, o resultado é:

Primeira falha - $C_0 = 69$, $F_0 = 76$

Candidatos sobreviventes

0A	0C	13	15
A2	BD	EE	F1

O 13 verdadeiro está entre eles, mas os outros sete também explicam essa primeira falha.

Precisamos repetir o experimento.

Made with Goodnotes

Segunda injeção de falha

Agora provocamos outra falha de um bit, no mesmo byte da entrada do último SubBytes.

Desta vez, vamos alterar o segundo bit:

$$e2 = 02$$

A entrada correta continua sendo BD, mas a defeituosa será:

$$BD \oplus 02 = Bf$$

Depois da S-Box e do último AddRoundKey, obtemos:

$$f_{2,0} = 1B$$

O segundo ciphertext defeituoso é:

1BC4E0D86A7B0430D8CDB78070B4C55A

Repetimos a busca de candidatos com:

$$C_0 = 69,\ f_{2,0} = 1B$$

Obtemos:

Segunda falha - candidatos sobreviventes

13	24	56	61
Af	B8	CA	DD

Agora fazemos a interseção das duas listas.

$$L_{final} = L_1 \cap L_2$$

Candidato fazemos a interseção das duas listas → 13, K10[0] recuperado!

Nesse exemplo, duas falhas foram suficientes para identificar unicamente o primeiro byte de K10.

No curso, Giraud é apresentado com cerca de 2 a 15 candidatos após um par correto / defeituoso e, geralmente, um único candidato após quatro pares. Nosso exemplo particular convergiu com dois pares.

Como recuperar os 16 bytes de K10?

Agora fazemos a mesma coisa para cada um dos 16 bytes do State na entrada do último SubBytes.

Cada byte é atacado separadamente, usando observações de falhas de um bit.

K10 do exemplo oficial AES-128

13	E3	F3	4D
11	94	07	2B
1D	4A	A7	30
7F	17	8B	C5

Depois de recuperar os 16 bytes, temos:

K10 = 13111D7FE3944A17F307A78B4D2B30C5

Como obter a chave original k0?

Usamos o Inverse Key Expansion do AES-128:

$$k10 \rightarrow k9 \rightarrow \cdots \rightarrow k0$$

Por exemplo, a partir das quatro palavras de K10:

w40 = 13111D7f

w41 = E3944A17

w42 = F307A78B

w43 = 4D2B30C5

Made with Goodnotes

Recuperamos:

$w39 = w43 \oplus w42$

$w38 = w42 \oplus w41$

$w37 = w41 \oplus w40$

$w36 = w40 \oplus g(w39)$

Assim obtemos k9:

549932D1F08557681093ED9CBE2C974E

Repetimos até chegar a:

K0 = 000102030405060708090A0B0C0D0E0F

Essa é a chave original do AES-128.

Giraud no SM4

Qual a ideia?

No AES com Giraud, fizemos:

Falha de 1 bit → Ciphertext defeituoso → Candidatos de K10

No SM4, queremos fazer:

Falha de 1 bit → Ciphertext defeituoso → Candidatos de rk_{31}

Depois, precisamos recuperar rk30, rk29 e rk28 para inventar o Key Expansion e encontrar a chave original.

Nosso exemplo SM4

Usaremos o mesmo vetor oficial:

Plaintext = 0123456789ABCDEFFEDCBA9876543210

Chave original = 0123456789ABCDEFFEDCBA9876543210

Ciphertext correto = 681EDF34D206965E86B3E94F536E4246

O ciphertext do SM4 possui quatro palavras:

$C = (X_{35}, X_{34}, X_{33}, X_{32})$

Logo:

X_{35} = 681EDF34

X_{34} = D206965E

X_{33} = 86B3E94F

X_{32} = 536E4246

Onde vamos injetar a falha?

A última rodada do SM4 calcula:

$X_{35} = X_{31} \oplus L(\tau(A))$

com:

$A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}$

Vamos provocar uma falha de um bit em um dos quatro bytes de A, imediatamente antes das S-Boxes da última rodada.

Made with Goodnotes

$X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}$

↓

Falha de 1 bit na entrada A

↓

τ - S-Box do SM4

↓

L - Rotação e XOR

↓

$\oplus X_{31}$

↓

X_{35} defeituoso

A hipótese é que somente A seja alterado. As palavras X_{31}, X_{32}, X_{33} e X_{34} permanecem iguais nas duas execuções.

Essa hipótese permite analisar a falha matematicamente, mas precisa ser validada em uma implementação física.

Exemplo real: uma falha de 1 bit

No nosso vetor de teste, a entrada correta dos S-Boxes da última rodada é:

$A = 96ff9d45$

Esse valor é conhecido pelo simulador para validar o experimento. O atacante não o conhece.

Vamos alterar o primeiro byte com XOR 01:

$C = 96\ ff\ 9d\ 45$

$F = 01\ 00\ 00\ 00$

$A' = 97\ ff\ 9d\ 45$

Passagem pelas S-Boxes

Byte	Entrada correta	Entrada com falha	Saída correta	Saída com falha
0	96	97	1A	55
1	FF	FF	48	48
2	9D	9D	8C	8C
3	45	45	73	73

Portanto:

$B = 1A488C73$

$B' = 55488C73$

$\Delta B = 4F000000$

A transformação L espalha o erro

Depois do S-Box, o SM4 aplica:

$L(B) = B \oplus (B <<< 2) \oplus (B <<< 10) \oplus (B <<< 18) \oplus (B <<< 24)$

Como a transformação L é linear, podemos aplicá-la diretamente à diferença:

$\Delta X_{35} = L(\Delta B)$

Vamos calcular com $\Delta B = 4F000000$.

$4F000000 \oplus 3C000001 \oplus 00000013C \oplus 00013C00 \oplus 004F0000 = \Delta X_{35} = 734E3D3D$

Made with Goodnotes

Qual o ciphertext defeituoso?

C = 681EDF34 D206965E 86B3E94F 536E4246

Como a falha ocorreu na última rodada, somente X_{35} muda.

Calculamos:

$$X'_{35} = X_{35} \oplus \Delta X_{35}$$

$$681EDF34 \oplus 734E3D3D = 1B50E209$$

Assim:

C = 681EDF34 D206965E 86B3E94F 536E4246

F = 1B50E209 D206965E 86B3E94F 536E4246

Δ = 734E3D3D 00000000 00000000 00000000

Até aqui simulamos a falha.

Agora somos o atacante: como descobrir a chave?

O atacante só recebeu:

C = 681EDF34 D206965E 86B3E94F 536E4246

C' = 1B50E209 D206965E 86B3E94F 536E4246

Ele calcula:

$$\Delta X_{35} = 734E3D3D$$

E sabe que:

$$\Delta X_{35} = L(\Delta B)$$

Portanto, desfaz a transformação linear:

$$\Delta B = L^{-1}(734E3D3D) = 4F000000$$

O que descobrimos?

Que a primeira S-Box produziu duas saídas cuja diferença é 4F.

Agora precisamos descobrir qual valor entrou nessa S-Box.

Como testar os candidatos?

No nosso primeiro experimento, sabemos que a falha foi XOR 01.

Então:

$$S(a) \oplus S(a \oplus 01) = 4F$$

Aqui, a é o byte desconhecido que entrou na S-Box.

O programa testa:

a = 00, 01, 02, ..., FF

Para cada valor, calcula a expressão e verifica se o resultado é 4F.

No nosso exemplo real, apenas dois valores passam:

Candidato 1	Candidato 2
96	97
$S(96) \oplus S(97) = 4F$	$S(97) \oplus S(96) = 4F$

Então conseguimos reduzir:

256 possibilidades → 2 candidatos

Mas qual deles é o correto?

Made with Goodnotes

Fazemos uma segunda falha

Executamos novamente o SM4 com o mesmo plaintext e a mesma chave.

Agora aplicamos uma falha XOR 02 ao mesmo byte de A.

Na simulação

$$96 \oplus 02 = 94$$

A S-Box transforma:

$$S(96) = 1A$$

$$S(94) = 9B$$

A diferença é:

$$1A \oplus 9B = 81$$

Depois da transformação L, obtemos:

$$L(81000000) = 85830606$$

O segundo ciphertext defeituoso é:

$$F2 = ED9DD932\ D206965E\ 86B3E94F\ 536E4246$$

O atacante calcula:

$$L^{-1}(C_0 \oplus F_{2,0}) = 81000000$$

E testa a nova equação:

$$S(a) \oplus S(a \oplus 02) = 81$$

Os candidatos são:

75, 77, 94, 96.

Cruzando as duas falhas

Agora temos duas listas:

Primeira falha - XOR 01

96 97

Segunda falha - XOR 02

75 77 94 96

Qual o valor aparece nas duas listas?

96

Encontramos a entrada original da primeira S-Box:

$$A[0] = 96$$

E agora podemos calcular o byte da subchave.

Como transformar 96 em um byte de rk31?

Lembre-se da equação da última rodada:

$$A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk31$$

Isolando a chave:

$$rk31 = A \oplus X_{32} \oplus X_{33} \oplus X_{34}$$

As três palavras X_{32}, X_{33} e X_{34} estão no ciphertext correto.

X_{32} = 536E4246

X_{33} = 86B3E94F

X_{34} = D206965E

XOR = 07DB3D57

Made with Goodnotes

Como recuperamos o primeiro byte de A (96), usamos o primeiro byte desse XOR (07):

$$96 \oplus 07 = 91$$

Primeiro byte de rk31 recuperado: 91

Como recuperar rk31 inteira?

Precisamos repetir o mesmo experimento para os outros três bytes de A.

Cada byte possui sua própria S-Box.

Byte	Entrada A verdadeira	Byte de $X_{32} \oplus X_{33} \oplus X_{34}$	Byte de rk31
0	96	07	91
1	FF	DB	24
2	9D	3D	A0
3	45	57	12

rk31 = 9124A012

A chave original

A chave original SM4 possui 128 bits, enquanto rk31 possui 32 bits.

Precisamos recuperar quatro subchaves consecutivas:

rk28, rk29, rk30, rk31

No nosso vetor oficial, elas são:

Subchave	Valor correto
rk28	428D3654
rk29	62293496
rk30	01CF72E5
rk31	9124A012

Como quatro subchaves viram a chave original?

Essa parte é o Inverse Key Expansion.

No SM4, cada subchave corresponde a uma palavra da expansão:

$$rk_i = K_{i+4}$$

Portanto:

$$rk_{28} = K_{32}$$
$$rk_{29} = K_{33}$$
$$rk_{30} = K_{34}$$
$$rk_{31} = K_{35}$$

A fórmula de expansão é:

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

Para inverter:

$$K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

Começamos calculando K_{31} a partir de K_{32}, K_{33}, K_{34} e K_{35}.

Depois calculamos K_{30}, K_{29} e assim por diante até K_0.

Quando chegamos a:

K_0, K_1, K_2, K_3

Made with Goodnotes

desfazemos as constantes iniciais Fk:

$$MK_i = K_i \oplus Fk_i$$

E obtemos a chave original:

0123456789ABCDEFFEDCBA9876543210

Made with Goodnotes